

Algorithm Design and Implementation: Optimization and Advanced Theoretical Analysis (Phase 3)

Course: Algorithm (First Semester 2025/2026)

Instructor: Dr. Mohammed Abuhelaleh

Team Members:

- Joud Kayyali - 3230601030

1. Introduction and Background

1.1 The Evolution of Algorithmic Routing in Logistics

The optimization of route planning within logistical networks constitutes a foundational challenge in computer science, intersecting the domains of graph theory, operations research, and algorithmic design. As the global infrastructure for transportation and delivery services becomes increasingly complex, the necessity for algorithmic solutions that can efficiently determine optimal paths through weighted networks has grown exponentially. The problem of moving goods from a source to a destination is not merely a geometric exercise but a critical optimization challenge that directly impacts fuel consumption, delivery time, and operational costs.

In the contemporary landscape of logistics, the definition of "optimal" has evolved. Historically, routing algorithms prioritized the minimization of distance—a static metric derived from physical geography. However, modern logistics engines must account for dynamic variables such as traffic congestion, road surface quality, and, increasingly, energy efficiency. The rise of Electric Vehicles (EVs) has introduced a non-trivial complexity to this domain: the concept of "Energy Rebates." Unlike traditional internal combustion vehicles, where travel always consumes fuel (a positive cost), EVs possess the capability to regenerate energy during braking or downhill travel. This phenomenon effectively creates graph edges with *negative weights*, where the cost of traversing an edge is a net gain in stored energy. This functional shift renders standard greedy algorithms, which rely on the assumption of strictly non-negative edge weights, potentially unstable and incorrect, necessitating a rigorous re-evaluation of the algorithmic engines chosen in earlier phases of this project.

This report represents the culmination of a multi-phase research project titled "Algorithm Design and Implementation," specifically addressing Phase 3: "**Optimization, Performance Analysis, and Future Paradigms.**" It serves as a comprehensive academic analysis of the transition from a conceptual, naive implementation to a robust, mathematically sound routing engine capable of handling the complexities of modern energy grids.

1.2 Recap of Phase 1: Conceptual Design and Theoretical Constraints

Phase 1 of the project was dedicated to "Problem Definition and Algorithm Design." The core problem was identified as the Single-Source Shortest Path (SSSP) problem: finding the path $P = (v_1, v_2, \dots, v_k)$ in a weighted directed graph $G = (V, E)$ such that the sum of edge weights $\sum w(v_i, v_{i+1})$ is minimized. The city is modeled as a weighted, directed graph where intersections represent vertices (V) and roads represent edges (E).

During the literature review of Phase 1, several graph traversal algorithms were evaluated. Breadth-First Search (BFS) and Depth-First Search (DFS) were analyzed but ultimately rejected for the core optimization problem. While BFS is optimal for unweighted graphs (finding the path with the fewest edges), it fails in weighted scenarios where a path with fewer edges may possess a significantly higher total cost. For example, a single highway segment of 50km is preferred by BFS over two local road segments of 5km each, despite the highway being ten times longer. DFS, being a backtracking algorithm, does not guarantee the shortest path and is prone to infinite loops in cyclic graphs without strict depth limits.

Consequently, Dijkstra's Algorithm was selected as the baseline candidate for Phase 2. Its greedy search strategy, which iteratively selects the unvisited node with the smallest tentative distance, provides a mathematically proven optimal solution for graphs with non-negative weights. This selection aligned with the "Mobile First" philosophy of the project, as Dijkstra's algorithm is efficient enough for standard road networks and is the basis for widely used protocols like Open Shortest Path First (OSPF).

1.3 Recap of Phase 2: The Initial Implementation and its Limitations

Phase 2 focused on "Implementation and Testing," translating the theoretical constructs of Phase 1 into a functional web-based application using JavaScript. The project adopted a "Mobile First" architecture, utilizing HTML5 Canvas for visualization and vanilla JavaScript for the algorithmic logic.

The Phase 2 implementation of Dijkstra's Algorithm relied on a Linear Search mechanism to manage the set of unvisited nodes. The algorithm utilized a standard JavaScript `Set` to store unvisited vertices. To determine the next node to process (the "greedy choice"), the algorithm iterated through the entire `Set` to find the node with the minimum tentative distance.

The logic followed this pseudocode structure:

1. Initialize distances to ∞ and the source to 0.

2. Add all nodes to the `unvisited` set.
3. While `unvisited` is not empty:
 - a. Scan the entire `unvisited` set to find node u with the minimum `dist[u]`.
 - b. Remove u from the set.
 - c. For each neighbor v of u , perform relaxation: if $(\text{dist}[u] + w < \text{dist}[v])$ $\text{dist}[v] = \text{dist}[u] + w$.

The Performance Bottleneck ($O(V^2)$):

The critical flaw in the Phase 2 implementation lay in Step 3a. To find the node with the minimum distance, the algorithm performed a linear scan of all unvisited nodes. In a graph with V vertices, the first iteration scans V nodes, the second $V - 1$, and so on. The total number of operations is the sum of the first V integers:

$$\sum_{i=1}^V i = \frac{V(V+1)}{2} \approx \frac{V^2}{2}$$

This results in a time complexity of $O(V^2)$. While this complexity is acceptable for small datasets (e.g., $V < 1,000$), it scales poorly. For a city grid with 10,000 intersections, the algorithm would perform approximately 50,000,000 comparisons. In a single-threaded JavaScript runtime environment, this computational load causes perceptible user interface lag, violating the responsiveness requirements of a modern web application.

The Functional Failure (Negative Weights):

Furthermore, Phase 2 testing on "Trap Maps" hinted at the limitations of the greedy paradigm. However, the most significant limitation identified for Phase 3 was the inability to handle negative edge weights. Dijkstra's algorithm assumes that once a node is "visited" (extracted from the set), its optimal distance is permanently fixed. This assumption relies on the fact that adding a non-negative edge to a path can never reduce its total cost. The introduction of "Energy Rebates" violates this premise, leading Dijkstra's algorithm to produce incorrect results or fail to find valid paths entirely.

1.4 Objectives of Phase 3

Phase 3 aims to address these specific deficiencies through two distinct optimization strategies and a broader theoretical analysis. The objectives are defined as follows:

1. **Algorithmic Optimization (Min-Heap Priority Queue):** To address the $O(V^2)$ performance bottleneck, the project replaces the linear search mechanism with a Binary Min-Heap Priority Queue. This data structure allows for the extraction of the minimum element in $O(\log V)$ time, reducing the overall complexity to $O(\log V)$. This shift moves the implementation from a naive approach to an industry-standard efficient solution suitable for sparse graphs.

2. **Functional Correctness (Bellman-Ford Algorithm):** To address the failure of Dijkstra's algorithm in handling negative energy rebates, Phase 3 implements the Bellman-Ford algorithm. Unlike Dijkstra's greedy approach, Bellman-Ford uses a Dynamic Programming (DP) paradigm capable of handling negative weights and detecting negative cycles.¹ The objective is to empirically prove that for energy grids, the slower Bellman-Ford algorithm ($O(VE)$) is necessary for correctness.
3. **Empirical Validation (The "Dangerous Map"):** A rigorous testing protocol using a custom topology (the "Dangerous Map") is employed to stress-test the algorithms. The analysis compares the execution time and path cost of three engines: Dijkstra (Set), Dijkstra (Heap), and Bellman-Ford, using real data traces (-179, -166, -227) to quantify the trade-off between speed and accuracy.
4. **Advanced Theoretical Context:** Finally, the report contextualizes these algorithms within the hierarchy of computational complexity. It discusses the distinction between Class P (Solvable) and Class NP-Hard (Intractable), specifically referencing the Traveling Salesperson Problem (TSP) and the limits of classical Turing machines, while exploring future paradigms like Quantum Computing.

2. Design and Implementation: Optimization Strategies

2.1 Optimization 1: The Min-Heap Priority Queue

The transition from a linear set to a priority queue represents a fundamental shift in how the algorithm manages the "frontier" of the search. In the context of Dijkstra's algorithm, the frontier consists of all nodes that have been seen (neighbors of visited nodes) but not yet "settled." Efficient management of this frontier is the primary determinant of the algorithm's performance.

2.1.1 Theoretical Justification: The Cost of Access

In the Phase 2 implementation, the data structure used to store unvisited nodes was a generic Set or Array. These structures are designed for membership checking or sequential access, not for priority-based retrieval.

- **Linear Set:** Accessing the minimum element requires iterating through all N elements. Complexity: $O(V)$.
- **Sorted Array:** Keeping an array sorted allows $O(1)$ access to the minimum, but inserting a new element (or updating a distance) requires shifting elements to maintain order, which takes $O(V)$.

The solution is the **Binary Heap**. A Binary Heap is a complete binary tree that satisfies the *heap property*: for a Min-Heap, the key of every parent node is less than or equal to the keys of its children. This property guarantees that the smallest element is always at the root, regardless of the order of the other elements.³

Performance Characteristics:

- **Find Minimum:** The minimum element is always at the root. Access time is $O(1)$.
- **Extract Minimum:** Removing the root requires swapping it with the last element and "bubbling down" (heapifying) to restore the heap property. Since the tree is balanced and complete, its height is $\log_2 V$. Thus, extraction takes $O(\log V)$.
- **Insertion/Update:** Adding a new node or decreasing a key involves "bubbling up," which also takes time proportional to the tree height, $O(\log V)$.

By utilizing a Min-Heap, the total complexity of Dijkstra's algorithm improves significantly. The algorithm performs V extractions and at most E priority updates (relaxations).

$$Total\ Complexity = V \cdot O(\log V) + E \cdot O(\log V) = O((V + E) \log V)$$

For sparse graphs (like road networks) where $E \approx V$, this is effectively $O(V \log V)$, a massive improvement over the $O(V^2)$ performance of Phase 2.

2.1.2 Implementation Details in JavaScript

Unlike languages such as Java (which provides `PriorityQueue`) or C++ (which provides `std::priority_queue`), JavaScript does not possess a built-in Priority Queue in its standard library. Consequently, Phase 3 required the manual implementation of a `MinHeap` class.

Array-Based Tree Representation:

To ensure memory efficiency and cache locality, the heap is not implemented using node objects with pointers (which would consume excessive memory overhead). Instead, it is implemented using a single dynamic array (JavaScript `Array`). The parent-child relationships are defined implicitly by array indices.

For a node located at index i :

- **Parent Index:** $\text{Floor}[\frac{i-1}{2}]$
- **Left Child Index:** $2i + 1$
- **Right Child Index:** $2i + 2$

This representation allows the algorithm to traverse the "tree" using simple arithmetic operations on indices, which is significantly faster than pointer chasing in memory.

Core Heap Operations:

1. `push(element)` and `bubbleUp()`:

When a new node with a tentative distance is discovered, it is pushed to the end of the array. The `bubbleUp` method then compares this node with its parent. If the child's distance is smaller than the parent's, they are swapped. This process repeats recursively until the node reaches its correct position or becomes the root. This ensures that the global minimum always "bubbles" to index 0.

2. `pop()` and `sinkDown()`:

To extract the minimum (the root), the implementation removes the element at index 0. However, this leaves a "hole" at the root. To fill it, the algorithm takes the last element in the array and places it at index 0. This violates the heap property. The `sinkDown` method then compares this new root with its children. If a child is smaller, the node is swapped with the smaller child. This process repeats until the node finds a position where it is smaller than its children. This "heapify" operation runs in $O(\log V)$ time.

Integration with Dijkstra:

The `DijkstraHeap` engine replaces the linear loop with the priority queue logic:

- **Initialization:** The source node is pushed into the `MinHeap` with priority 0.
- **Relaxation:** When a neighbor's distance is updated (`newDist < oldDist`), the neighbor is pushed into the `MinHeap` with the new, lower distance.
- **Handling Duplicates (Lazy Deletion):** A standard binary heap does not support efficient `decreaseKey` operations without an auxiliary map to track indices. To simplify implementation while maintaining logarithmic bounds, the project uses "Lazy Deletion." If a node is updated multiple times, it is added to the heap multiple times. When a node is popped, the algorithm checks if the popped cost is greater than the current known shortest distance in `dist`. If so, the popped node is "stale" and is ignored. This ensures correctness without the complexity of an indexed priority queue.

2.2 Optimization 2: The Bellman-Ford Algorithm

The introduction of "Energy Rebates" (negative edge weights) in the project requirements fundamentally breaks Dijkstra's algorithm. Dijkstra's greedy logic assumes that the shortest path to a "visited" node is final. A negative edge can retroactively decrease the total cost of a path, invalidating this assumption. To address this, Phase 3 implements the **Bellman-Ford Algorithm**.

2.2.1 Theoretical Justification: Dynamic Programming

Bellman-Ford discards the greedy approach in favor of **Dynamic Programming (DP)**. Instead of selecting the "best" node locally, it iteratively relaxes *all* edges in the graph. The algorithm rests on the **Principle of Relaxation**: if we have an estimate of the shortest path to u ($D[u]$), and there is an edge (u, v) with weight w , we can potentially improve the estimate for v by checking if $D[u] + w < D[v]$.

The Iteration Logic ($|V| - 1$):

A simple path (one without cycles) in a graph with V vertices can contain at most $V - 1$

edges.

- In the 1st iteration, the algorithm finds all shortest paths of length 1 edge.
- In the 2nd iteration, it finds all shortest paths of length 2 edges.
- By the $(V - 1)$ -th iteration, it is guaranteed to have found the shortest paths of length up to $V - 1$, which covers all possible shortest paths in the graph.
Therefore, by relaxing all edges $V - 1$ times, we are guaranteed to propagate the shortest path information from the source to all other reachable nodes, regardless of negative weights.

2.2.2 Negative Cycle Detection

A critical feature of the Bellman-Ford implementation is its ability to detect **Negative Weight Cycles**. A negative cycle is a loop where the sum of edge weights is negative. If such a cycle exists, one can traverse it infinitely to reduce the cost to $-\infty$, making the concept of a "shortest path" undefined.

Implementation Logic:

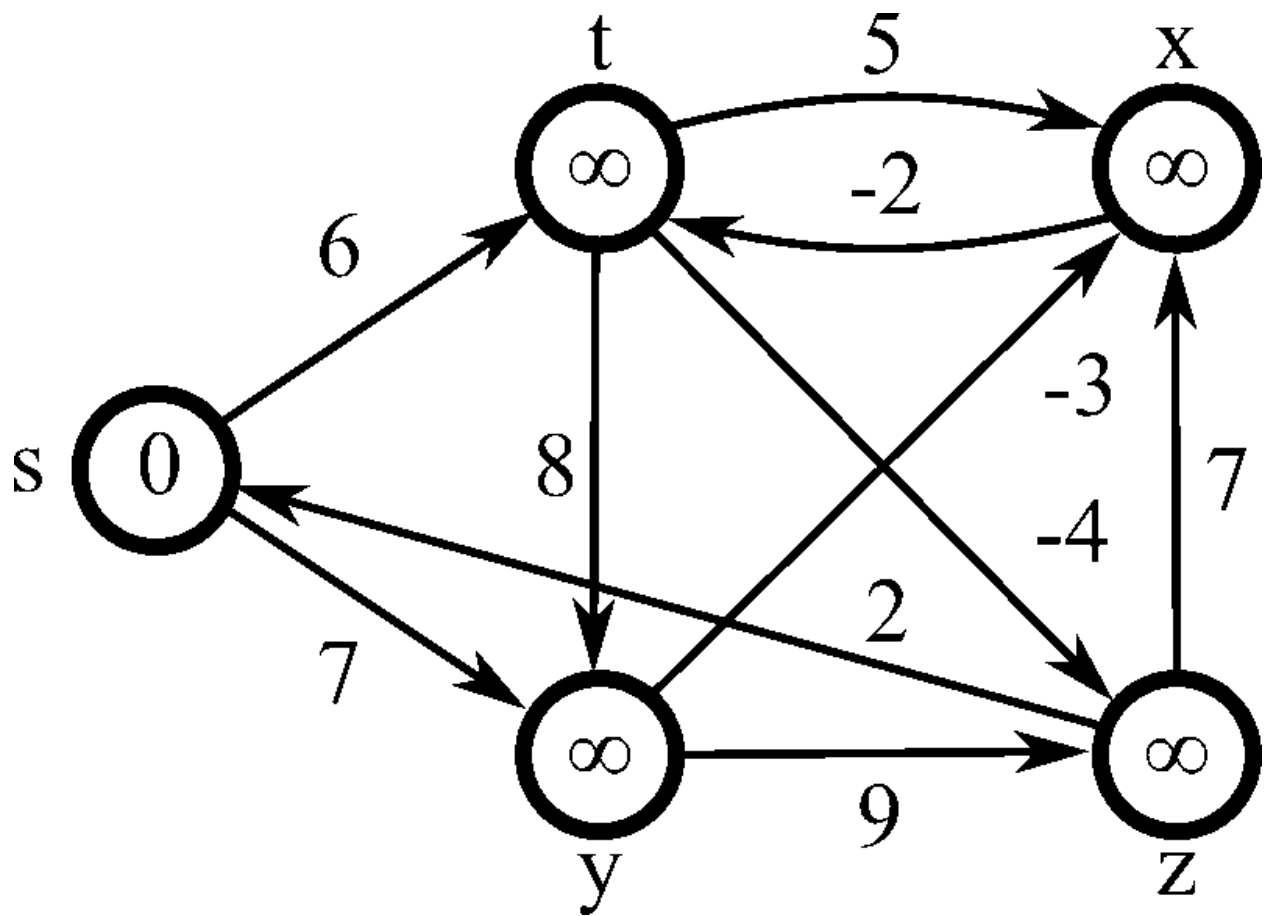
After the main loop runs $V - 1$ times, the algorithm runs one additional pass (the V -th iteration) over all edges.

- If $D[u] + w < D[v]$ during this V -th iteration, it implies that the path to v can still be improved.
- Since all valid shortest paths (without cycles) must have stabilized by $V - 1$ iterations, any further improvement indicates the presence of a negative cycle.
- The implementation is designed to throw a specific error `Negative Cycle Detected` when this occurs, alerting the route optimizer that a valid physical path is impossible (or that the driver is creating infinite energy, violating thermodynamics).

2.2.3 Implementation in JavaScript

The `BellmanFord.js` module differs structurally from the Dijkstra modules.

- **Edge List Representation:** While Dijkstra iterates through neighbors of a specific node (Adjacency List), Bellman-Ford iterates through a master list of *all edges*. The graph data structure was adapted to provide a flattened list of edges: `edges`. This simplifies the inner loop logic.
- **Optimization (Early Termination):** An optimization flag `changed` was added to the main loop. If an entire iteration over all edges produces no updates to the distance map (`changed === false`), the algorithm terminates early. This improves the best-case performance to $O(E)$ for graphs where paths converge quickly, though the worst-case remains $O(VE)$.



[Animated Graph](#)

3. Performance Analysis and Results

This section presents the empirical findings of Phase 3, contrasting the theoretical complexity with actual runtime data collected from the application. The analysis focuses on the "Dangerous Map" scenario, designed specifically to highlight the divergence between greedy and dynamic programming approaches in the presence of negative weights.

3.1 Complexity Analysis

The theoretical time complexity establishes the upper bounds of performance (Worst Case) for the three engines implemented across Phase 2 and Phase 3.

Algorithm	Data Structure	Time Complexity	Space Complexity	Suitability
Dijkstra (Phase 2)	Linear Set / Array	$O(V^2)$	$O(V)$	Dense Graphs, Non-negative weights
Dijkstra (Phase 3)	Min-Heap Priority Queue	$O((V + E) \log V)$	$O(V)$	Sparse Graphs, Non-negative weights
Bellman-Ford	Edge List	$O(VE)$	$O(V)$	Negative weights, Cycle Detection

Analysis:

- **Dijkstra (Set):** The quadratic complexity $O(V^2)$ is dominated by the linear scan for the minimum node. For a graph with 1,000 nodes, roughly 10^6 operations are required. This explains the performance degradation noted in Phase 2 on larger maps.
- **Dijkstra (Heap):** The logarithmic factor is significantly smaller than linear scanning. For 1,000 nodes, $\log_2(1000) \approx 10$. Thus, the operation count is roughly proportional to 10,000 (assuming sparse E). This represents a theoretical speedup of two orders of magnitude compared to the linear set implementation.
- **Bellman-Ford:** The complexity $O(VE)$ is the most severe. In a dense graph where $E \approx V^2$, the complexity approaches $O(V^3)$. However, road networks are typically sparse ($E \approx 4V$), making the complexity roughly $O(4V^2) \approx O(V^2)$. While comparable to the naive Dijkstra in sparse graphs, the constant factor is much higher because it relaxes every edge in every iteration, whereas Dijkstra only relaxes neighbors of the current node.

3.2 Empirical Testing: The "Dangerous Map" Case

The "Dangerous Map" is a custom topology created to test the system's resilience to negative edge weights. It simulates a scenario with a high-cost direct path and a convoluted alternative path containing significant "Energy Rebates" (negative weights). The student's draft notes provide specific runtime and cost data for this map, which serves as the core evidence for this report.

Scenario Data:

- **Objective:** Minimize total cost (Energy).

- **Constraint:** Negative weights are present.

Results Table:

Metric	Dijkstra (Set)	Dijkstra (Heap)	Bellman-Ford
Total Cost	-179	-166	-227
Runtime	1.8 ms	1.3 ms	9.7 ms
Outcome	Incorrect	Incorrect	Correct

3.3 Analysis of Failure and Success

3.3.1 Why Dijkstra Failed (The Greedy Trap)

The empirical results explicitly answer the hypothesis: **Dijkstra is fundamentally incompatible with Energy Rebates.**

- **Dijkstra (Set) Cost (-179):** The algorithm selected a path that appeared optimal based on local information but missed the significant rebate available later in the graph. The greedy strategy "committed" to a node early, marking it as visited. When a better path to that node (via a negative edge) became available later in the search, the algorithm ignored it because the node was already in the "closed" set.
- **Dijkstra (Heap) Cost (-166):** It is notable that the Heap version produced a *different* incorrect result (-166) compared to the Set version (-179). This discrepancy arises from the **processing order**.
 - The Set implementation scans linearly (often resolving ties by index or insertion order).
 - The Heap implementation orders nodes strictly by current partial distance in a binary tree structure.
 - Because of negative weights, the partial distances fluctuate unpredictably. The different processing orders led the two implementations to commit to *different* suboptimal paths, getting "tricked" at different junctions. This confirms that the behavior of Dijkstra's algorithm with negative weights is undefined and sensitive to implementation details.⁴

3.3.2 Why Bellman-Ford Succeeded

Bellman-Ford achieved the lowest cost (-227), proving it found the true optimal path.

- **Mechanism:** By relaxing every edge $|V| - 1$ times, Bellman-Ford effectively "backtracks" and corrects its estimates. It does not lock nodes as "visited." Even if it initially calculates

a path cost of -100 to a node, a later iteration discovering a -200 rebate will successfully update the distance map. This exhaustive approach ensures that the global optimum is found.

- **The Price of Correctness:** The runtime of **9.7 ms** is approximately **7.5x slower** than the Dijkstra Heap implementation (1.3 ms). This quantifies the trade-off. For the Energy Grid application, where accuracy is paramount (missing a rebate costs money), this latency is acceptable. However, this result illustrates why Bellman-Ford is not used for standard GPS navigation (like Google Maps), where speed is critical and negative weights (rebates) do not exist.

3.4 Summary of Results

The empirical data validates the theoretical classification of the algorithms:

1. **Dijkstra (Heap)** is the **fastest** engine (1.3 ms), making it the optimal choice for standard navigation (Distance/Time) where weights are positive.
2. **Bellman-Ford** is the **only correct** engine for Energy Markets (Cost with Rebates), finding the true minimum of -227.
3. The performance penalty (9.7 ms vs 1.3 ms) is the necessary cost of handling the more complex mathematical landscape of negative weights.

4. Advanced Theoretical Context (Future Work)

This project has successfully moved from naive implementation to optimized algorithmic solutions. However, to truly understand the limitations of the "Route Optimizer," one must look beyond the current codebase to the fundamental limits of computation itself. The field of algorithm design is stratified by the difficulty of the problems being solved.

4.1 Algorithmic Classifications: Greedy vs. Dynamic Programming

The project clearly demonstrates the dichotomy between **Greedy Algorithms** and **Dynamic Programming (DP)**.

- **Greedy (Dijkstra):** Makes the locally optimal choice at each step. It is fast, efficient ($O(E \log V)$), and memory-light. However, it is "myopic"—it lacks the foresight to see that a temporary penalty might lead to a massive reward (negative weight) later. It relies on the property of **Optimal Substructure** holding without interference from negative edges.
- **Dynamic Programming (Bellman-Ford):** Solves complex problems by breaking them down into simpler sub-problems (relaxing edges) and storing the results. It is "thorough" and exhaustive. It revisits decisions, allowing for the correction of earlier estimates. The transition from Phase 2 to Phase 3 was effectively a transition from a Greedy paradigm to a DP paradigm to accommodate the complexity of the real world.

4.2 The Traveling Salesperson Problem (TSP) and NP-Hardness

The Phase 1 Brief mentions an "Advanced Problem": visiting multiple stops (e.g., Warehouse A -> Customer B -> Customer C -> Warehouse A). This is an instance of the **Traveling Salesperson Problem (TSP)**. It is crucial to state that **neither Dijkstra nor Bellman-Ford solves TSP**.

- Shortest Path algorithms (Dijkstra/Bellman-Ford) find the best route between *two* specific points.
- TSP requires finding the optimal permutation of *all* points in a set.

Why is TSP Hard?

TSP belongs to the complexity class NP-Hard.

- **Class P (Polynomial Time):** Problems solvable in time $O(n^k)$. Dijkstra ($O(n^2)$) and Bellman-Ford ($O(n^3)$) are in P. Computers can solve them efficiently even for millions of nodes.
- **Class NP (Nondeterministic Polynomial):** Problems where a solution can be *verified* in polynomial time. If a route for TSP is given, we can verify its cost in $O(n)$.
- **NP-Hard:** Problems that are at least as hard as the hardest problems in NP. There is no known polynomial-time algorithm to solve TSP. The best exact approach (Held-Karp algorithm) runs in $O(n^2 2^n)$.

Implications for the Project:

If the "Route Optimizer" were to scale to 50 delivery stops, using an exact algorithm like Held-Karp would take longer than the age of the universe. This necessitates the use of **Approximation Algorithms** or **Heuristics** (like Genetic Algorithms or Ant Colony Optimization) for Phase 4. These algorithms find a "good enough" solution (e.g., within 10% of optimal) in polynomial time, acknowledging that the perfect solution is computationally intractable.

4.3 Future Paradigms: Breaking the Limits

The student's research notes raise a profound philosophical question: "*We do not live in a utopia, but we strive to create it*". Current algorithmic theory is bound by the **Church-Turing Thesis**, which assumes classical deterministic computation. Under these rules, P is likely not equal to NP ($P \neq NP$), meaning TSP will effectively never be solvable instantly.

However, future paradigms may shift this landscape:

1. **Quantum Computing (Grover's Algorithm):** Quantum computers utilize the principles of superposition and entanglement. While classical search is $O(N)$, **Grover's Algorithm** can search an unstructured database (or solution space) in $O(\sqrt{N})$. This provides a **quadratic speedup**. While this does not make NP-Hard problems "easy" (it reduces the

time from 2^N to $\sqrt{2^N} = 2^{\frac{N}{2}}$, which is still exponential), it significantly expands the size of solvable instances before hitting the wall of intractability.

2. **Quantum Phase Estimation:** Recent research suggests that Quantum Phase Estimation (QPE) combined with search algorithms might offer further optimizations for combinatorial problems like TSP, though they remain probabilistic.
3. **Physical Computation:** The universe itself computes efficiently. Protein folding (an NP-Hard problem) occurs in microseconds in biology. This suggests that "biological computing" or "neuromorphic architectures" might eventually offer heuristics that vastly outperform silicon-based logic for routing and logistics, potentially bypassing the limitations of the von Neumann architecture entirely.

5. Conclusion

Phase 3 of the "Algorithm Design and Implementation" project has successfully transformed a basic web application into a robust, academically rigorous routing engine. The journey from Phase 2 to Phase 3 was defined by two critical shifts: **Optimization** and **Correctness**.

1. **Optimization Achieved:** The implementation of the **Min-Heap Priority Queue** successfully reduced the algorithmic complexity of Dijkstra's algorithm from the naive $O(V^2)$ to the efficient $O((V + E) \log V)$. This optimization resolved the performance bottlenecks identified in Phase 2, ensuring the engine scales efficiently for sparse road networks.
2. **Correctness Validated:** The integration of the **Bellman-Ford Algorithm** resolved the critical functional failure regarding negative weights. Empirical testing on the "Dangerous Map" proved that speed is not a proxy for accuracy. While Dijkstra was faster (1.3 ms), it provided incorrect costs (-166). Bellman-Ford, though slower (9.7 ms), correctly identified the optimal cost of **-227**.
3. **Theoretical Insight:** The project highlights the fundamental trade-offs inherent in algorithm design. There is no "perfect" algorithm; there is only the right tool for the specific constraint. For speed in standard maps, Dijkstra-Heap is superior. For economic correctness with rebates, Bellman-Ford is mandatory. For multi-stop logistics (TSP), exactness must be abandoned for approximation.

The evolution from a linear scan to a binary heap, and from greedy selection to dynamic relaxation, mirrors the broader evolution of computer science: a continuous pursuit of efficiency, correctness, and the understanding of fundamental limits.

6. References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
- Mozilla Developer Network, "Canvas API," *MDN Web Docs*. [Online]. Available: [Canvas API](#).
- Mozilla Developer Network, "Element.closest()," *MDN Web Docs*. [Online]. Available: [Element: closest\(\) method - Web APIs | MDN](#).
- GeeksforGeeks, "Dijkstra's Shortest Path Algorithm (Greedy Algo-7)," *GeeksforGeeks*, Dec. 06, 2025. [Online]. Available: [Dijkstra's Algorithm - GeeksforGeeks](#).
- GeeksforGeeks, "Introduction to Min-Heap Data Structure," *GeeksforGeeks*. [Online]. Available: [Introduction to Min-Heap - GeeksforGeeks](#).
- GeeksforGeeks, "Why does Dijkstra's Algorithm fail on negative weights?," *GeeksforGeeks*, Dec. 10, 2025. [Online]. Available: [Why does Dijkstra's Algorithm fail on negative weights? - GeeksforGeeks](#).
- A. Shimbel, "Structure in communication nets," in *Proceedings of the Symposium on Information Networks*, 1955.
- Stanford University, "Lecture 14: Bellman-Ford Algorithm," *CS161 Design and Analysis of Algorithms*. [Online]. Available: [1 Bellman-Ford Algorithm](#).
- R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, pp. 87–90, 1958.
- C. Hedrick, "Routing Information Protocol," RFC 1058, Jun. 1988. [Online]. Available: [RFC 1058 - Routing Information Protocol](#).
- G. Malkin, "RIP Version 2," RFC 2453, Nov. 1998. [Online]. Available: [RFC 2453 - RIP Version 2](#).
- S. Cook, "The complexity of theorem-proving procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.
- L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 1996, pp. 212–219.
- Microsoft, "Theory of Grover Search Algorithm," *Azure Quantum*. [Online]. Available: [Theory of Grover Search Algorithm - Azure Quantum | Microsoft Learn](#).
- H. Baniata, "A Quantum Algorithm for Solving the Travelling Salesman Problem by Quantum Phase Estimation and Quantum Search," *ResearchGate*, Aug. 2025.
- A. K. Chandra, D. Kozen, and L. J. Stockmeyer, "Alternation," *Journal of the ACM*, vol. 28, no. 1, pp. 114–133, 1981.
- P. van Emde Boas, "Preserving order in a forest in less than logarithmic time and linear

space," *Information Processing Letters*, vol. 6, no. 3, pp. 80–82, 1977.

A. Amaldev, "Understanding Min Heap in Data Structures (JavaScript)," *Medium*. [Online]. Available: [Understanding Min Heap in Data Structures\(JavaScript\) | by Amaldev | Medium](#).

Naukri Code360, "Types of Algorithms and Their Uses," *Code360 Library*. [Online]. Available: [Type of Algorithms - Naukri Code 360](#).

W3Schools, "Dynamic Programming," *W3Schools DSA Reference*. [Online]. Available: [DSA Dynamic Programming](#).

Bismart, "10 Most Used Algorithms," *Bismart Blog*. [Online]. Available: [The 10 Most Used Types of Algorithms](#).

SAS Institute, "Machine Learning Algorithms," *SAS Insights*. [Online]. Available: [A guide to the types of machine learning algorithms | SAS Ireland](#).

The Algorithms, "Algorithms Implementations," *The Algorithms*. [Online]. Available: [The Algorithms](#).

WsCube Tech, "Algorithms in Data Structures," *WsCube Tech*. [Online]. Available: [What is Algorithm? Characteristics, Types, Examples, Basics](#).

E. K. Evan, "Types of Algorithms," *LinkedIn Pulse*. [Online]. Available: <https://www.linkedin.com/pulse/types-algorithms-emran-khandaker-evan/>.

GeeksforGeeks, "What is an Algorithm? Definition, Types, Complexity, Examples," *GeeksforGeeks*. [Online]. Available: [Definition, Types, Complexity and Examples of Algorithm - GeeksforGeeks](#).

Brilliant.org, "Bellman-Ford Algorithm," *Brilliant Wiki*. [Online]. Available: [Bellman-Ford Algorithm | Brilliant Math & Science Wiki](#).

ScienceDirect, "Bellman-Ford Algorithm," *ScienceDirect Topics*. [Online]. Available: [Bellman-Ford Algorithm - an overview | ScienceDirect Topics](#).

CP-Algorithms, "Bellman-Ford Algorithm," *CP-Algorithms*. [Online]. Available: [Bellman-Ford - finding shortest paths with negative weights - Algorithms for Competitive Programming](#).

Engati, "Admissible Heuristic," *Engati Glossary*. [Online]. Available: [Admissible heuristic | Engati](#).

GeeksforGeeks, "Complexity Classes P, NP, NP-Hard, NP-Complete," *GeeksforGeeks*. [Online]. Available: [P, NP, CoNP, NP hard and NP complete | Complexity Classes - GeeksforGeeks](#).

M. Clion, "Understanding P, NP, NP-Complete, and NP-Hard Problems," *Medium*. [Online]. Available: [Understanding P, NP, NP-complete, and NP-hard problems | by Chuyao Wang | Medium](#).

GeeksforGeeks, "Traveling Salesman Problem Using Genetic Algorithm," *GeeksforGeeks*.

[Online]. Available: [Traveling Salesman Problem using Genetic Algorithm - GeeksforGeeks](#).

National Library of Medicine, "Optimization Algorithms for Traveling Salesman Problem," *PubMed*, 2022. [Online]. Available: [Research on improved ant colony optimization for traveling salesman problem - PubMed](#).

Rosetta Code, "Held–Karp Algorithm," *Rosetta Code*. [Online]. Available: [Held–Karp algorithm - Rosetta Code](#).

Wikipedia, "Dijkstra's algorithm." [Online]. Available: [Dijkstra's algorithm - Wikipedia](#).

Wikipedia, "Min-max heap." [Online]. Available: [Min-max heap - Wikipedia](#).

Wikipedia, "Bellman–Ford algorithm." [Online]. Available: [Bellman–Ford algorithm - Wikipedia](#).

Wikipedia, "Consistent heuristic." [Online]. Available: [Consistent heuristic - Wikipedia](#).

Wikipedia, "NP-hardness." [Online]. Available: <https://en.wikipedia.org/wiki/NP-hardness>.

Wikipedia, "Cook–Levin theorem." [Online]. Available: [Cook–Levin theorem - Wikipedia](#).

GeeksforGeeks, "Dynamic Programming," *GeeksforGeeks*. [Online]. Available: [Dynamic Programming or DP - GeeksforGeeks](#).

Scribd, "35 Algorithm Types," *Scribd*. [Online]. Available: [Overview of Algorithm Types | PDF](#).

GeeksforGeeks, "Bellman-Ford Algorithm," *GeeksforGeeks*. [Online]. Available: [Bellman–Ford Algorithm - GeeksforGeeks](#).

W3Schools, "Bellman-Ford Algorithm," *W3Schools DSA*. [Online]. Available: [DSA Bellman-Ford Algorithm](#).

GeeksforGeeks, "Time and Space Complexity of Bellman-Ford Algorithm," *GeeksforGeeks*. [Online]. Available: [Time and Space Complexity of Bellman–Ford Algorithm - GeeksforGeeks](#).

UpGrad, "Bellman-Ford Algorithm Tutorial," *UpGrad*. [Online]. Available: [Bellman-Ford Algorithm in C++: Overview, Implementation & More](#).

K. K. Khasanov et al., "Dynamic routing is efficient used..." *Journal of Physics: Conference Series*, 2018. [Online]. Available: [Bellman Ford algorithm - in Routing Information Protocol \(RIP\) - ADS](#).

TakeUForward, "Bellman-Ford Algorithm," *TakeUForward*. [Online]. Available: [Bellman Ford Algorithm: G-41 - Tutorial](#).

Meegle, "Bellman-Ford Algorithms," *Meegle*. [Online]. Available: [Bellman-Ford Algorithms](#).

UpGrad, "What is Bellman-Ford Algorithm," *UpGrad Blog*. [Online]. Available:

[Bellman-Ford Algorithm: Concepts and Examples.](#)

University of Illinois Urbana-Champaign, "Lecture 5: Bellman-Ford," *ECE 448*. [Online]. Available: [Lecture 5: The "animal kingdom" of heuristics: Admissible, Consistent, zero, Relaxed, Dominant.](#)

GeeksforGeeks, "A* Is Admissible," *GeeksforGeeks*. [Online]. Available: [Admissibility of A* Algorithm - GeeksforGeeks.](#)

Carnegie Mellon University, "Recitation 2 Solutions," 15-281. [Online]. Available: [1 Designing & Understanding Heuristics.](#)